

Technical Document Evaluation Reos
Course: CMPS 4113: Software Engineering

Draft

Format: Scale 1-10 4

- ☒ Title page /logo.
- ☒ Revision History
- ☒ Table of Contents no page #s.
- ☒ Font/ Spacing/ Margins consistent
- ☒ Use of Headers Doesn't look great
- ☒ Section names and numbers proper
- ☒ Every section has text
- ☒ Captions and descriptions for figures and tables
- ☒ Necessary citations/references
- ☒ Page numbering 5

Content Scale 1-10 5

- ☐ Introduction
 - ☒ Motivation/purpose/Scope need work
 - ☒ Overview of document
- ☒ All relevant material/complete use case diagram
- ☒ Organization on
- ☒ Creative
- ☒ Correct spelling/grammar phrasing - strange.

Name/Group Mouse Trap / Team
Semester: Spring 2023 Something Good?

Final Grade: 12/20

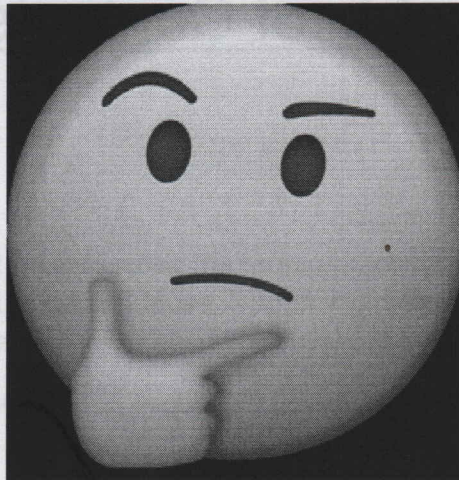
Format Scale 1-10 7

- ☒ Title page
- ☒ Revision History
- ☒ Table of Contents
- ☒ Font/ Spacing/ Margins consistent
- ☒ Use of Headers
- ☒ Section names and numbers proper
- ☒ Every section has text
- ☒ Captions and descriptions for figures and tables
- ☒ Necessary citations/references?
- ☒ Page numbering

Content Scale 1-10 5

- ☒ Introduction
 - ☒ Motivation/purpose/Scope
 - ☒ Overview of document
- ☒ All relevant material/complete use case diagram
- ☒ Organization several scenarios
- ☒ Creative OOA Diagram
- ☒ Correct spelling/grammar was mistakes

Garcia
De la Cruz
Dillon
Espino
Norstrom



Requirements Documentation

Team SomethingGood?

Kyle Garcia, Alex Dela Cruz, Zach Dillion, Jose Espino, Sierra Norstrom

Version 2
3/05/2026

Table of Contents

1.) <u>Introduction</u>	3
1.1) Purpose	
1.2) Scope	
1.2.1.) Main Objective	
1.2.2.) Specific Goals	
1.3.) Overview of Document	
2.) <u>System</u>	4
2.1.) Target Environment	
2.2.) Who are the Users	
2.3.) Functional Requirements	
2.3.1.) Major System Functions	
2.3.2.) Minor System Functions	
2.3.3.) Major Classes Identified	
2.4.) Non-Functional Requirements	
2.4.1.) Technical	
2.4.2.) Performance	
2.4.3.) Usability	
3.) <u>Project Constraints and Risks</u>	5
3.1.) Schedule Constraints	
3.2.) Management Risks	
4.) <u>UML Diagrams</u>	6

1. Introduction

1.1 Purpose

The purpose of this document is to describe the requirements for a board game style application inspired by the mechanics of the game Mouse Trap. It will define the scope of the project and the functional requirements necessary to ensure a successful prototype

and non-functional

1.2 Scope

1.2.1 Main Objective

The objective of this project is to develop a functional prototype of a strategy game. The application focuses on the digital translation of "trap" mechanics into a software environment, prioritizing grid-based movement, and a turn-management system.

1.2.2 Specific Goals

- Grid Infrastructure: The game uses a 2D top-down view with 32 x 16 hexagonal grid.
- Player Roles: The game supports two different roles: one Trapper and one to four Mice
- Trapper Victory Condition: The Trapper wins by deactivating grid cells to block all paths between Mouse players and the board's outer edges.
- Mouse Victory Condition: Mouse Players win by moving across the grid to reach any "Edge" cell (outermost row or column of the 32 x 16 grid).
- System Logic & Navigation: The application will use a centralized "state machine" (a controller) to manage two levels of logic:
 - Application States (Game Manager): To handle transitions between Main Menu, Game Customization, Active Gameplay, and the Game Over screen.
 - Turn States (Turn Manager): To manage the flow of movement within the game, ensuring the Trapper and Mice follow a strict turn-based sequence and validating all moves against the grid's "active" button status.

yes - you did great on format here! why didn't you do this in plan?

1.3 Overview of Document

This document provides a technical description of the project's scope and functional requirements. It outlines the hardware and software dependencies necessary to run the application and details the specific behavior of the application for both the Trapper and Mouse roles. Additionally, it defines the system logic used to manage the game's various states and player turns.

There are sub-goals no need language!

2. System

2.1 Target Environment

The project will be developed and executed in the standard Windows 11 computing environment.

Hardware:

- Desktop or Laptop PC

Software:

- Operating System: Windows 11
- Development Environment: Visual Studio Code
- Language: Python 3.10.11
- Library: Pygame
 - Used for its sprite class to manage mouse player rendering and layering
 - Used for the Event Queue to capture MOUSEBUTTONDOWN triggers
 - Used for Drawing Primitives to render the custom hexagonal geometry

2.2 Who Are the Users

The target users are employees of the customer's company looking for morale-boosting entertainment during scheduled downtime. The application supports two distinct playstyles:

- The Trapper: A strategic role focused on area denial and path blocking.
- The Mouse: A tactical role focused on navigation and escape

use case?

2.3 Functional Requirements:

2.3.1 Major System Functions

VER B5.

- Grid Infrastructure: The system will maintain a 32 X 16 hexagonal grid.
- Custom Interaction: The system shall use trigonometry to draw the shape of the button while radial distance math is used for the actual interaction of when the button is clicked. Since standard rectangular buttons aren't fit for our envisioned gameboard
- Game Manager: A centralized state machine that handles high-level transitions between the Main Menu, Game Customization, Active Gameplay, and Game Over screens.
- Turn Management: While in the "Active Gameplay" state, the system manages a secondary turn-based state machine. This ensures a strict sequence of play, alternating between the Trapper's wall placement and each Mouse player's movement, while validating those actions only occur during the correct player's turn.
- Win/Loss Detection: The system will automatically detect when a Mouse reaches the board perimeter or when all paths are obstructed.

[These are subsystems]

2.3.2 Minor System Functions

- Turn Indicator: A visual display showing which player is currently active.
- Sprite Rendering: Using the Pygame Sprite class to ensure the Mouse icon remains on the top visual layer of the grid. *Render Sprite*

2.3.3 Major Classes Identified

the major classes are -

- Hexagon Button: A custom UI class that handles its own coordinates, color states, and click-detection logic.
- Gameboard: The primary data structure that initializes and stores the 32 x 16 grid. It acts as the intermediary, receiving requests from players and updating the specific Hexagon Button states
- Trapper: A logic class that calculates strategic "wall" placement. It sends a "wall" request to the Gameboard based on user input coordinates.
- Mouse: A class inheriting from pygame.sprite.Sprite. It sends "Move" requests to the Gameboard, which then validates if the target hexagon is "Active" before allowing the sprite to update its position.
- Game Manager: The High-level controller that manages the state machine. It decides when to switch between the Main Menu, Game Customization, Active Gameplay, and Game Over screens.

2.4 Non-Functional Requirements

2.4.1 Technical

The project will be developed and must be run in a Windows 11 system. The project will be written in Visual Studio Code with Python utilizing PyGame for GUI features. ✓

2.4.2 Performance

The project should promptly respond to user input with little delay. It should also be user friendly so that new players can quickly learn and play the game.

2.4.3 Usability

The Interface shall be intuitive, allowing new players to understand the movement and trapping mechanics without an external manual

3. Project Constraints and Risks

3.1 Schedule Constraints

The project is subject to a semester-long development lifecycle. Due to this time limit, the primary focus is the delivery of a minimal requirement product. This prototype will prioritize core mechanic stability such as 32 x 16 grid logic and turn-based state machine over advanced graphical polish.

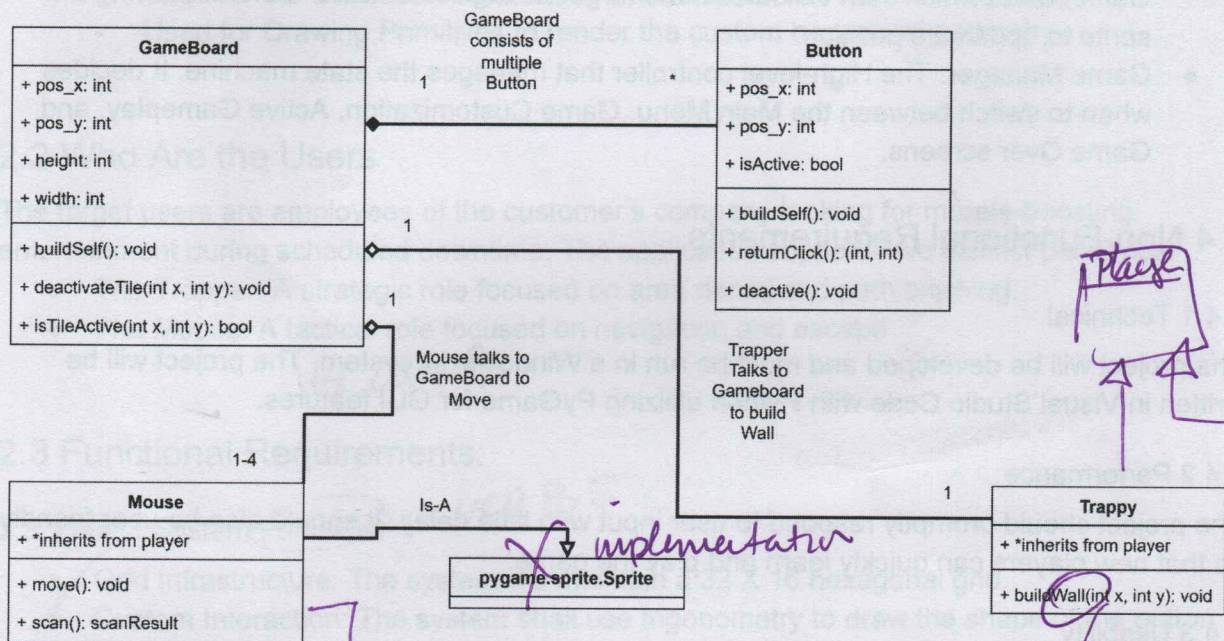
Language?

3.2 Management Risks

With a team of five members, coordination and version control are identified as primary risks. To mitigate this, the project utilizes a modular architecture (separating button.py, mouse.py, and main.py) to allow for simultaneous development without file conflicts. We're also using Github Codespace to further minimize this risk. Codespace ensures that all members are working on identical dependencies and provides a centralized "Source of Truth" for the codebase, significantly reducing integration errors during the final assembly.

4. UML Diagrams

OOA Diagram



Gameboard Has A (strong) Button - Gameboard consists of hexagon buttons made from Button class

Gameboard Has A (weak) Mouse - Gameboard has 1-4 mouse player

Gameboard Has A (weak) Trapper - Gameboard has a Trapper

Mouse inherits from pygame.sprite.Sprite - Mouse class uses pygame.sprite.Sprite from pygame library

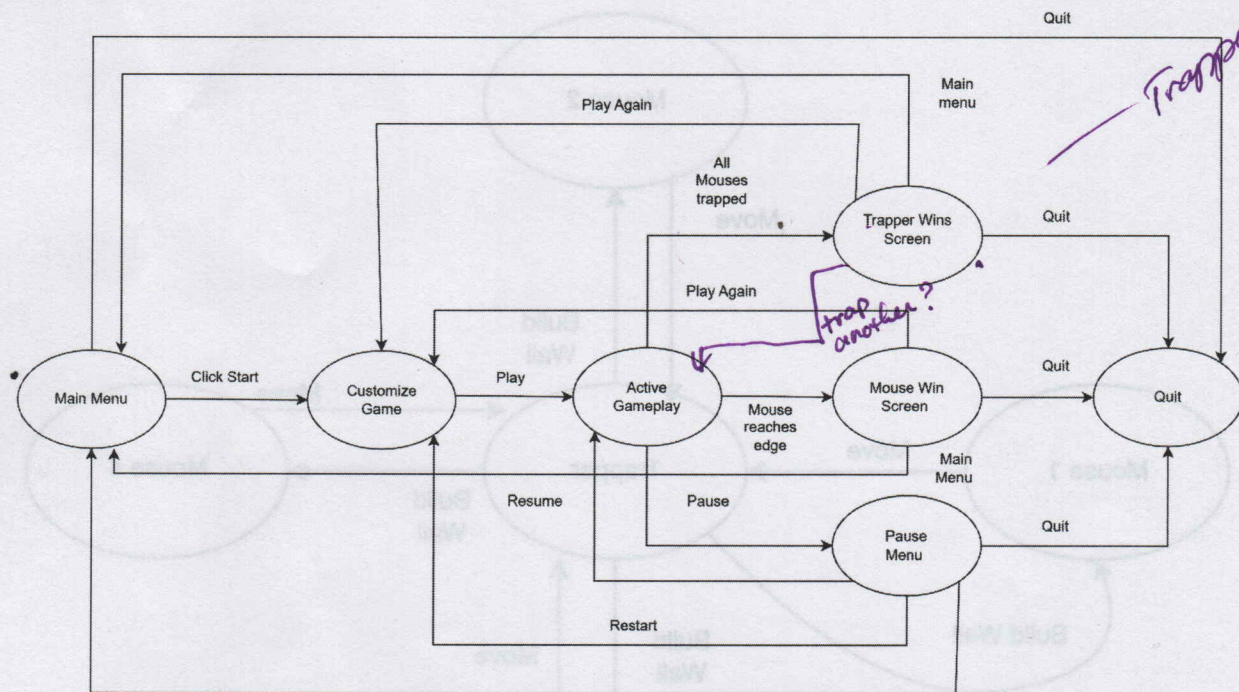
so wall or class?

Design Diagram

Team Something Good?

7

GUI State Machine Diagram



Trapper Wins Screen

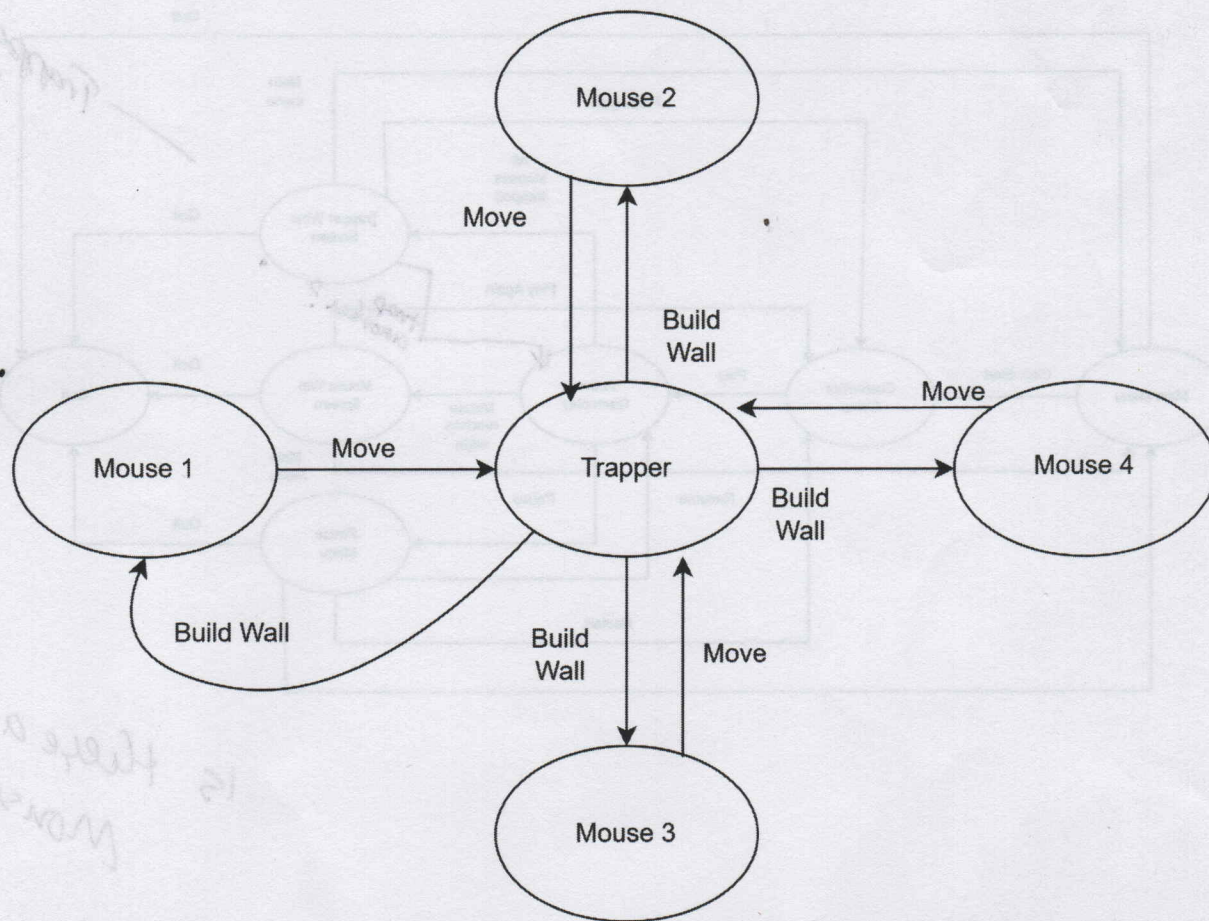
trap another?

Is there a mouse lose screen??
8/10.

v.l.

Reverse if so.
if not
Resubmit with
note.

Player Turns State Machine Diagram



Pattern is basically alternating turns between Mouse and Trapper.

Example: Mouse 1, Trapper, Mouse 2, Trapper, Mouse 3, Trapper, Mouse 4, Trapper
Then it goes back to Mouse 1 repeating the process.

This will be dynamic based on how many players are playing. The diagram is based on 5 players 1 Trapper 4 Mouse.